

1. Project Title: Personal Protective Equipment (PPE) Detection using YOLOv8

2. Project Overview / Problem Selection

The objective of this project is to develop a computer vision-based Personal Protective Equipment (PPE) detection system using the YOLOv8 object detection model. The system is designed to detect workers and identify whether required safety equipment, particularly hardhats and safety vests, is present or absent.

The system detects the following five classes:

- Person
- Hardhat
- Safety Vest
- NO-Hardhat
- NO-Safety Vest

The project focuses on workplace safety monitoring and can help identify PPE compliance issues in environments where workers are required to wear protective equipment.

3. Tools and Technologies Used

Tool / Technology Purpose	
Python	Programming and model implementation
Google Colab	Development and training environment
YOLOv8n	Object detection model
Ultralytics	YOLOv8 implementation
Roboflow	Dataset management and annotation
Google Drive	Dataset and model storage
PyTorch	Deep learning framework
OpenCV	Image processing and inference support
Matplotlib	Visualization of results

Tool / Technology Purpose

Pandas

Results analysis

4. Dataset Preparation

The dataset used for this project was prepared for detecting Personal Protective Equipment (PPE) in workplace environments. The dataset contains images of workers in construction and industrial settings, including different PPE conditions.

The project uses five object classes: Person, Hardhat, Safety Vest, NO-Hardhat, and NO-Safety Vest. The original dataset contained a total of 1,387 images, divided into 967 training images, 284 validation images, and 136 test images.

For the manual annotation requirement, a separate subset of images was prepared and annotated using Roboflow. The manually annotated subset contains 99 image-label pairs, organized into 69 training images, 20 validation images, and 10 test images. The annotations were created in YOLO format with bounding boxes around the relevant objects.

The dataset was stored in Google Drive and connected to Google Colab for model training, evaluation, and inference.

5. Data Annotation

The images were manually annotated using Roboflow. Bounding boxes were drawn around the relevant PPE-related objects visible in each image, and the appropriate class was assigned to each bounding box.

The annotation process focused on five classes: Person, Hardhat, Safety Vest, NO-Hardhat, and NO-Safety Vest. The bounding boxes were kept as close as possible to the boundaries of the objects to improve the quality of the training data.

Special attention was given to workers with and without required PPE. For example, workers wearing helmets and safety vests were annotated with the corresponding PPE classes, while missing PPE was represented using the NO-Hardhat and NO-Safety Vest classes according to the project's annotation scheme.

The manually annotated dataset contains 99 image-label pairs. The annotations were exported in YOLO format and used as part of the dataset preparation and analysis process.

6. Dataset Organization and YOLO Format

The dataset was organized into three main subsets: training, validation, and testing. The original dataset contained 967 training images, 284 validation images, and 136 test images. This

organization provided separate data for model training, validation during development, and final performance evaluation.

The dataset followed the YOLO object detection format. Each image had a corresponding .txt annotation file containing the class ID and normalized bounding-box coordinates. The annotation format follows:

```
class_id center_x center_y width height
```

The coordinate values are normalized between 0 and 1 relative to the image dimensions. This format allows YOLOv8 to efficiently process the annotated images during training.

The manually annotated subset was also organized into train, valid, and test folders, containing 69, 20, and 10 image-label pairs respectively.

7. Dataset Configuration (data.yaml)

A data.yaml configuration file was used to define the dataset paths and object classes for YOLOv8 training and evaluation. The configuration specified separate directories for the training, validation, and test images.

The dataset contains five object classes. The class configuration used in the project is:

- **Class 0** — Person
- **Class 1** — Hardhat
- **Class 2** — Safety Vest
- **Class 3** — NO-Hardhat
- **Class 4** — NO-Safety Vest

The data.yaml file specifies the dataset structure and the total number of classes (nc: 5). YOLOv8 uses this configuration file to locate the images and labels and to understand the classes that the model needs to detect.

The dataset configuration was successfully used during both model training and test evaluation.

8. Google Colab Setup

Google Colab was used as the development and model training environment for this project. Colab provided a convenient Python-based environment with GPU support, which helped accelerate the YOLOv8 training process.

The dataset and project files were stored in Google Drive and accessed from the Colab notebook. The required Python libraries, including Ultralytics, were installed in the Colab environment before training.

A Tesla T4 GPU was used for model training. The YOLOv8n model was loaded using the Ultralytics framework, and the dataset configuration file (data.yaml) was provided to the model for training and evaluation.

Google Colab was also used for test evaluation, generating prediction images, visualizing training results, and performing error analysis.

9. Model Training

The YOLOv8n (YOLOv8 Nano) object detection model was selected for this project because it provides a good balance between detection performance and computational efficiency.

The model was trained using the prepared PPE dataset in Google Colab. The training process was performed for 30 epochs with an image size of 640×640 pixels. A Tesla T4 GPU was used to accelerate the training process.

During training, the model learned to detect the five PPE-related classes: Person, Hardhat, Safety Vest, NO-Hardhat, and NO-Safety Vest. Training and validation metrics were monitored throughout the training process to evaluate the model's learning progress.

The best-performing model weights were saved as best.pt and stored in Google Drive for later testing and inference.

The training process completed successfully, and the generated results included training/validation loss curves, precision-recall curves, F1 curves, and a confusion matrix.

10. Model Evaluation

After training, the trained YOLOv8n model was evaluated on the test dataset to measure its object detection performance. The evaluation was performed using Precision, Recall, mAP@0.5, and mAP@0.5:0.95.

The model was evaluated on 136 test images containing 1,210 annotated object instances. The obtained results were:

Metric	Result
Precision	81.55%
Recall	57.07%
mAP@0.5	64.25%
mAP@0.5:0.95	38.70%

The Precision of 81.55% indicates that most of the objects detected by the model were relevant detections. The Recall of 57.07% shows that the model detected a moderate proportion of the

objects present in the test images. The $mAP@0.5$ of 64.25% demonstrates reasonable detection performance at an IoU threshold of 0.5, while the $mAP@0.5:0.95$ of 38.70% reflects the stricter evaluation across multiple IoU thresholds.

The evaluation results show that the trained model can detect PPE-related objects, although there is still room for improvement, particularly in reducing missed detections and improving bounding-box accuracy.

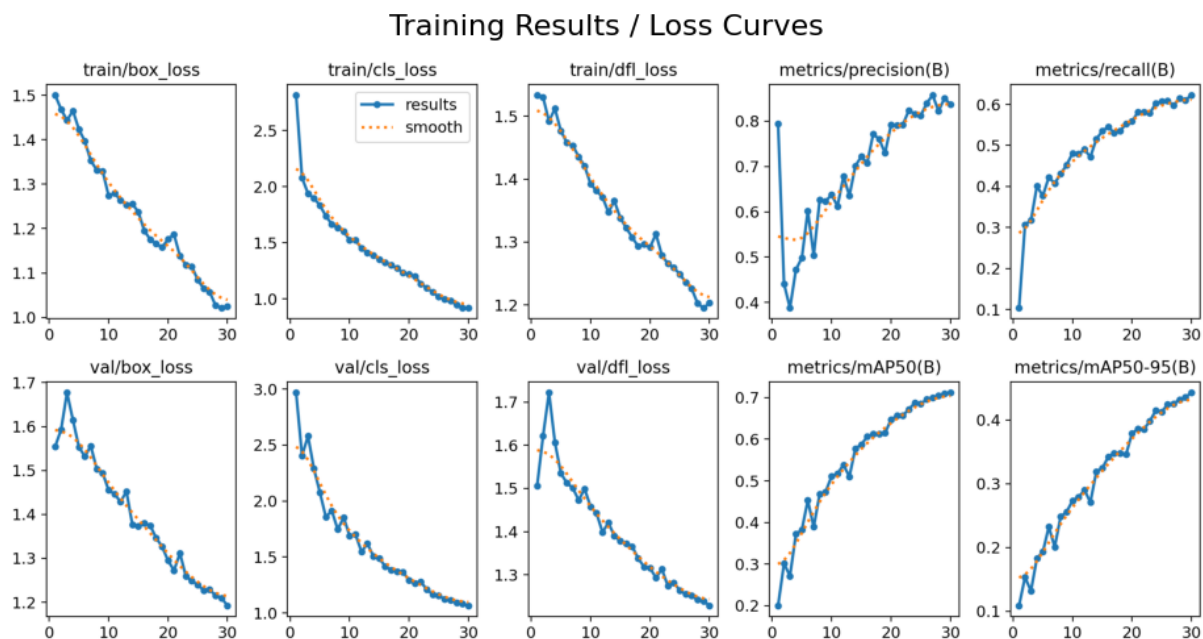


Figure 1: Training and Validation Results of the YOLOv8n PPE Detection Model

The training and validation results show the performance of the YOLOv8n model during training. The graphs illustrate how the model's loss and evaluation metrics changed across the training epochs. These results indicate the learning progress and overall performance of the PPE detection model.

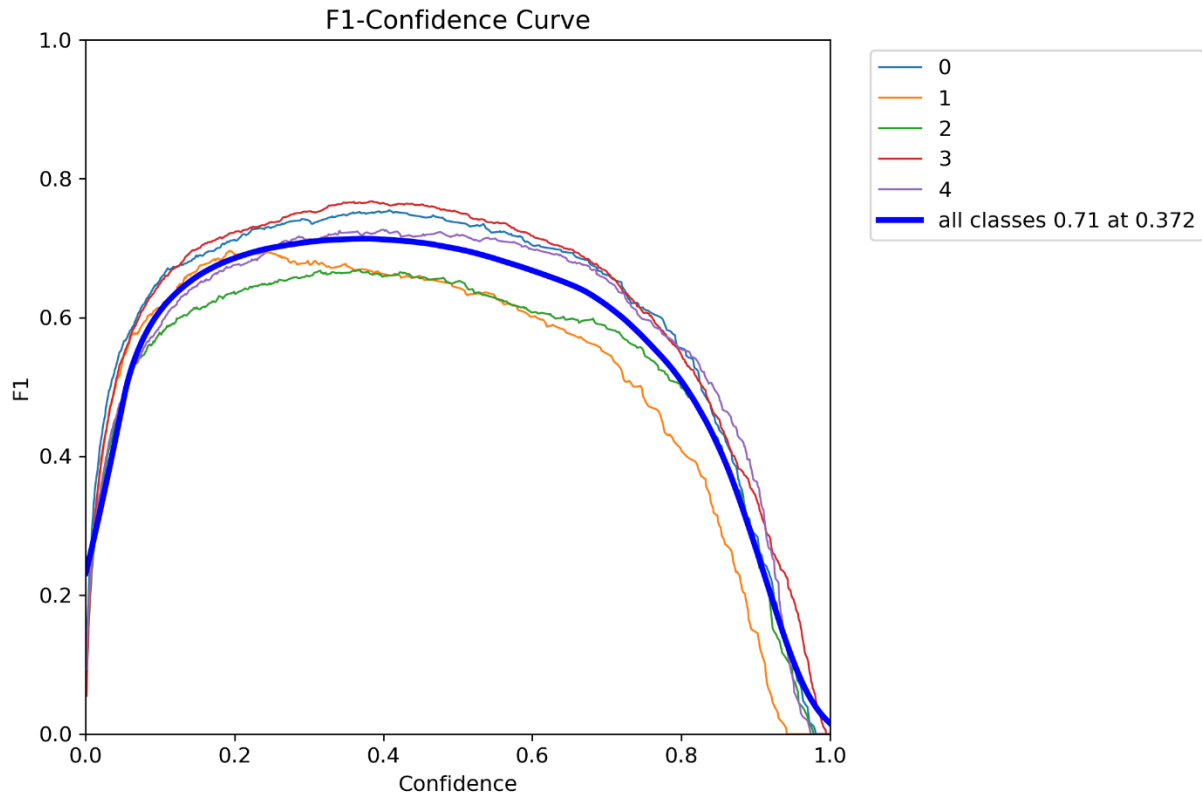


Figure 2: F1 Curve of the YOLOv8n PPE Detection Model

The F1 curve shows the balance between precision and recall at different confidence thresholds. A higher F1-score indicates better overall detection performance of the model. The curve helps identify the confidence level at which the model provides a good balance between precision and recall.

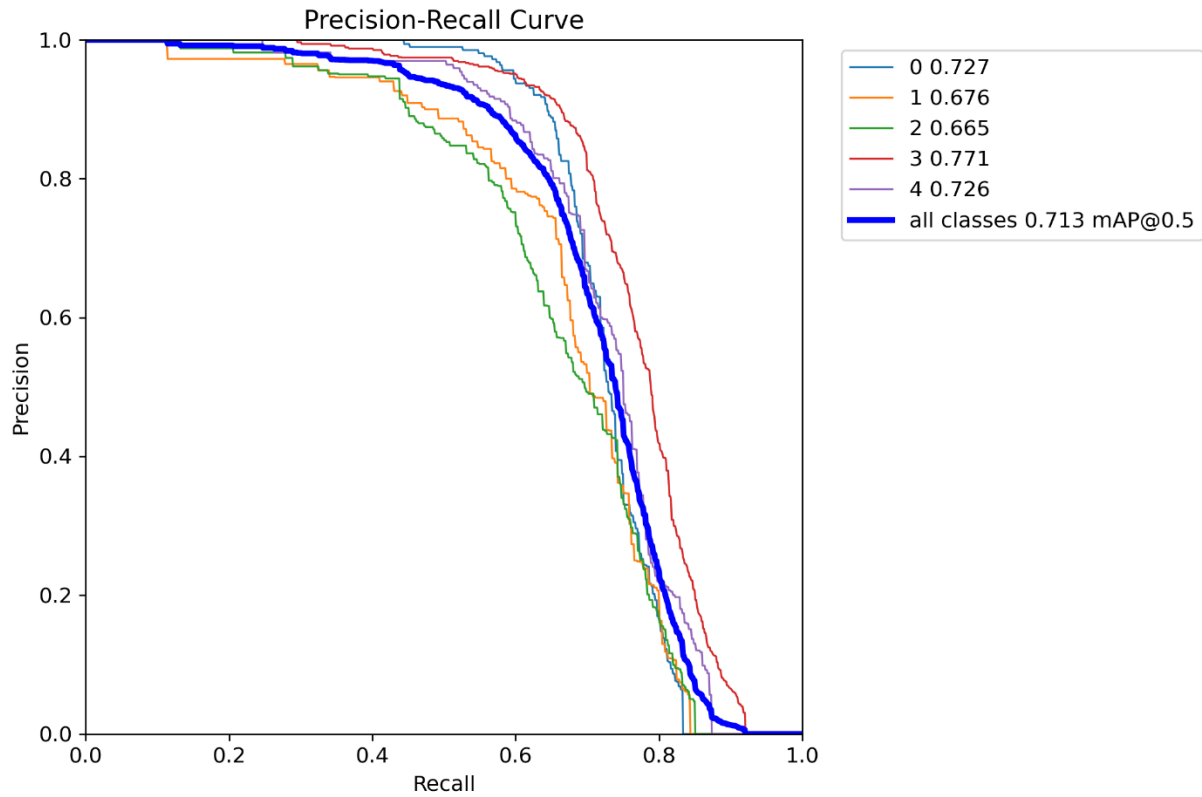


Figure 3: Precision–Recall Curve of the YOLOv8n PPE Detection Model

The Precision–Recall curve illustrates the relationship between precision and recall for the five PPE-related object classes. The curve helps evaluate how effectively the model detects the target classes while minimizing false detections.

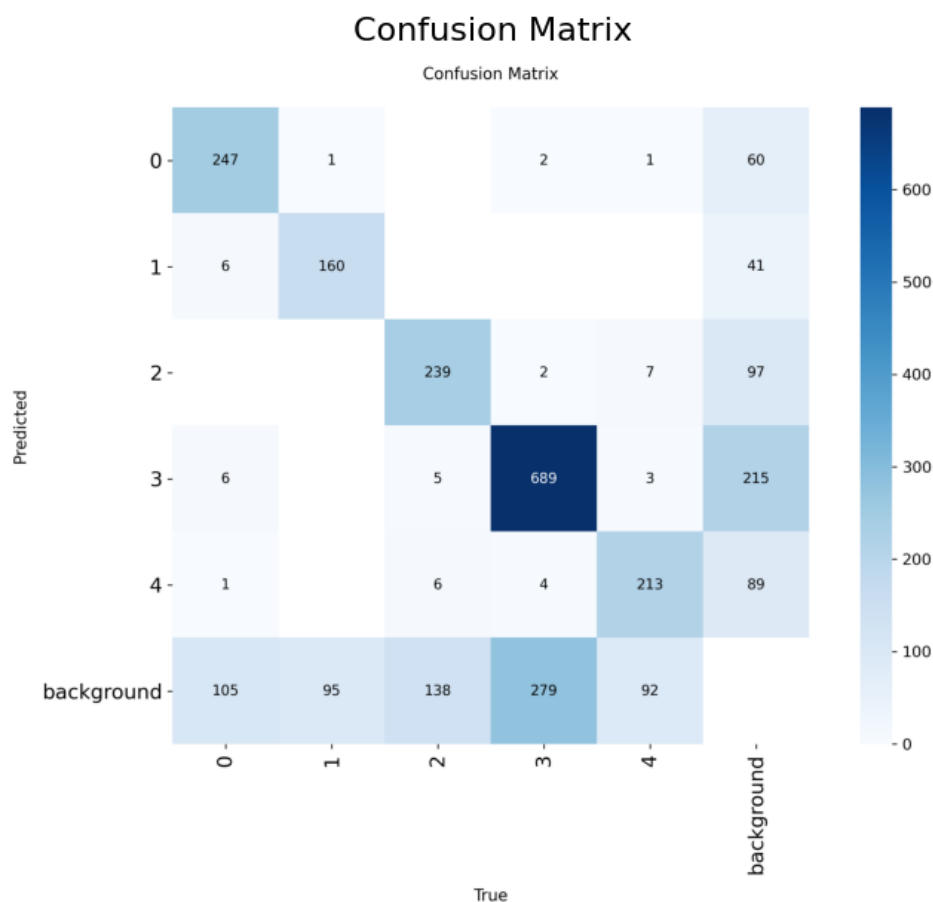


Figure 4: Confusion Matrix of the YOLOv8n PPE Detection Model

The confusion matrix shows the classification performance of the model across the five object classes. It helps identify correctly classified objects as well as cases where one class is confused with another.

11. Inference Results

After completing the model training and evaluation, the trained best.pt model was used to perform inference on test images. The purpose of inference was to verify how the trained model performs on unseen images and to visualize its object detection results.

The model generated bounding boxes around detected objects and displayed the predicted class names along with confidence scores. The inference results included detections for the five PPE-related classes: Person, Hardhat, Safety Vest, NO-Hardhat, and NO-Safety Vest.

More than 15 prediction images were generated and saved from the inference process. These results provided a visual demonstration of the model's ability to identify PPE-related objects in workplace images.

The inference results also helped identify some weak predictions and over-detection cases, which were further reviewed during the Error Analysis stage.

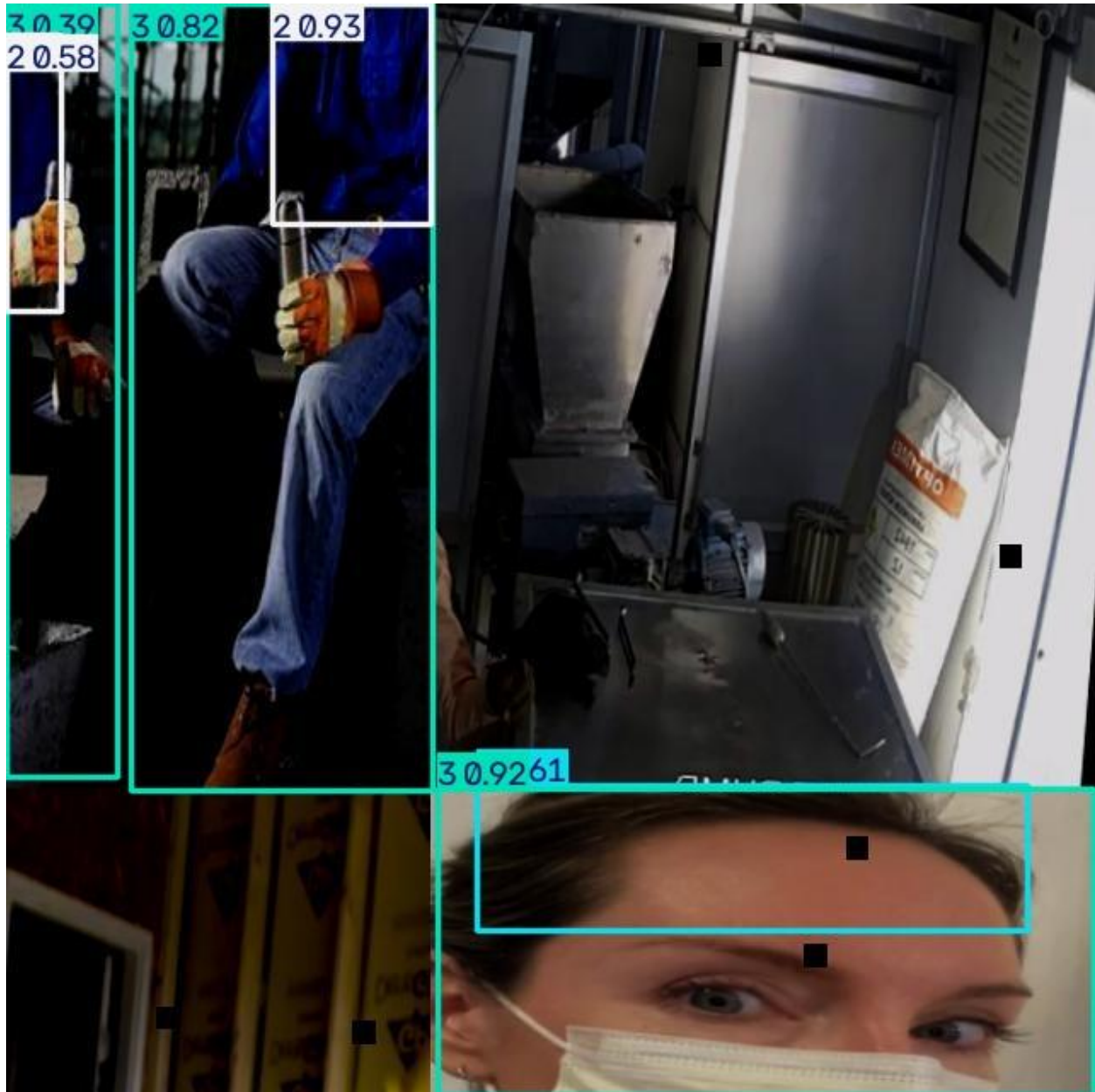


Figure 5: YOLOv8n PPE Detection Prediction on a Test Image

12. Error Analysis

Error analysis was performed on 10 test images to identify cases where the model produced weak or inconsistent predictions. The predicted class counts were compared with the ground-truth annotations to identify possible over-detection, missed detections, and wrong-class predictions.

The analysis showed that the most common issue was over-detection, where the model predicted more objects than were present in the ground-truth annotations. This was particularly noticeable in images 022, 062, 079, and the Pexels-Yankrukov image. Some predictions also included classes that were not present in the corresponding ground-truth annotations.

One image (076) showed a possible wrong-class prediction, where the ground truth contained Person and NO-Safety Vest, while the model predicted NO-Hardhat detections. Image 010 showed relatively consistent prediction counts compared with its ground truth.

Error Analysis Summary

Image	Observed Issue
010	Prediction counts were consistent with ground truth
014	Possible false positives / over-detection
018	Possible false positives / extra predictions
022	Severe over-detection
057	Extra predictions / possible false positives
062	Strong over-detection
076	Possible wrong-class prediction and missed classes
079	Severe over-detection
Pexels-Purvesh	Extra classes and one possible missed vest
Pexels-Yankrukov	Severe over-detection

The analysis suggests that the model may have difficulty with complex scenes, visually similar PPE regions, and images containing multiple workers or small objects. Some errors may also be influenced by annotation inconsistencies.

Possible Improvements

1. Increase the amount and diversity of manually annotated training data.
2. Improve the consistency and accuracy of bounding-box annotations.
3. Include more challenging images with small objects, crowded scenes, and different lighting conditions.
4. Train the model for more epochs and experiment with confidence thresholds.
5. Experiment with a larger YOLO model, such as YOLOv8s, if additional computational resources are available.

6. 13. What I Learned

7. Through this project, I gained practical experience in developing an object detection system using YOLOv8. I learned how to prepare an object detection dataset, manually annotate images, organize images and labels in YOLO format, and configure a dataset using a data.yaml file.
8. I also learned how to use Roboflow for dataset management and annotation and Google Colab for training and evaluating a deep learning model using GPU resources.
9. The project helped me understand important object detection evaluation metrics such as Precision, Recall, mAP@0.5, and mAP@0.5:0.95. I also learned how to analyze confusion matrices, training results, prediction images, and model errors.
10. Most importantly, I learned that good dataset quality, accurate annotations, and diverse training images are important for achieving reliable object detection performance. Performing error analysis also helped me understand the limitations of a trained model and identify areas for improvement.

14. Future Improvements

Although the developed PPE detection system achieved reasonable detection performance, several improvements can be made in future work.

1. **Increase Dataset Size:** More diverse and high-quality images can be collected and annotated to improve the model's ability to generalize to different workplace environments.
2. **Improve Annotation Quality:** More consistent and precise bounding-box annotations can help reduce incorrect detections and improve localization accuracy.
3. **Increase Training Epochs:** Training the model for more epochs may allow it to learn more complex visual patterns and improve detection performance.

4. **Use Data Augmentation:** Techniques such as image rotation, scaling, cropping, brightness adjustment, and flipping can increase dataset diversity and improve model robustness.
5. **Experiment with Larger Models:** Larger YOLOv8 variants, such as YOLOv8s or YOLOv8m, can be tested to determine whether they provide better accuracy.
6. **Improve Difficult-Case Detection:** Additional images containing small PPE objects, crowded scenes, poor lighting, and partially visible workers can be included to reduce false positives and missed detections.
7. **Real-Time Video Detection:** The system can be extended to process live CCTV or workplace video streams for continuous PPE compliance monitoring.

8. 15. Conclusion

9. This project successfully demonstrated the development of a Personal Protective Equipment (PPE) detection system using the YOLOv8n object detection model. The system was trained to detect five classes: Person, Hardhat, Safety Vest, NO-Hardhat, and NO-Safety Vest.
10. The model was trained for 30 epochs using Google Colab and evaluated on 136 test images. It achieved a Precision of 81.55%, Recall of 57.07%, mAP@0.5 of 64.25%, and mAP@0.5:0.95 of 38.70%.
11. The project also included manual dataset annotation, YOLO-format dataset preparation, model evaluation, inference, and error analysis. The results demonstrate that YOLOv8 can be used for automated PPE monitoring, although further improvements in dataset quality, training, and model optimization could increase its accuracy and reliability.
12. Overall, this project provided practical experience in building an end-to-end computer vision object detection system and demonstrated the potential of AI-based solutions for improving workplace safety monitoring.

16. Deliverables and Project Resources

The following deliverables were prepared as part of the project:

- Google Colab notebook containing the complete implementation and outputs.
- YOLO-format dataset with images and corresponding annotation files.
- data.yaml dataset configuration file.
- Trained YOLOv8n model weights (best.pt).
- Training results, including loss curves and performance graphs.
- Confusion matrix and evaluation results.
- More than 15 prediction images generated during inference.

- Error analysis of 10 test images.
- Final project report documenting the complete workflow and results.

The project files and trained model were stored in Google Drive. The relevant project links can be added below:

Google Drive Link:

https://drive.google.com/drive/folders/1BMgS4j7IR1_UQ0jdO10bToaoZgQLH6SU?usp=sharing

Google Colab Notebook Link:

<https://colab.research.google.com/drive/1drs9WVVZWzkhLbQpJqTWDJ09vYTTZSPu?usp=sharing>

GitHub Repository Link: <https://github.com/cheemaawais465-sudo/PPE-Detection-YOLOv8>